LINUX ADMINISTRATION PORTFOLIO

Linux User and Group Management

Project 1 — Practical command reference and learning notes

Environment: Ubuntu 26.04 LTS on WSL2 • Bash

Scenario: Create Alex and Maria as members of an IT group and configure a shared workspace.

This guide records the commands practised during the project, explains the important options, and summarises the results. Passwords and other secrets are intentionally not included.

1. Project goal and final result

The goal was to practise local Linux account and group administration, directory ownership, permissions, setgid, and collaboration between two ordinary user accounts.

Command / concept	Purpose and notes
`Group`	IT Supplementary group for the shared workspace.
`Users`	alex and maria Each user has their own home directory and private primary group.
`Shared directory`	/srv/it-share Shared workspace for members of IT.
`Directory owner:group`	alex:IT Alex owns the directory; IT is its owning group.
`Directory mode`	2770 / drwxrws--- Owner and group have rwx; others have no permissions; setgid is enabled.
`Final group entry`	IT:x:1003:alex,maria Both users appear as supplementary members of IT.

Important: The shared directory's setgid bit makes new items inherit the directory's group. Group write access to new files also depends on the file-creation mode and the user's umask; setgid alone does not guarantee it.

2. Creating the project workspace

```
mkdir -p ~/linux-learning/portfolio/linux-user-group-management
```

mkdir creates directories. The **-p** option creates missing parent directories and does not complain if the target directory already exists.

```
ls -l ~/linux-learning/portfolio/
```

```
ls -la ~/linux-learning/portfolio/linux-user-group-management/
```

ls -l shows a detailed listing. **ls -a** also shows hidden entries, including . and ... These were used to check that the project folder existed and was empty before its README was created.

3. Creating the IT group and Alex

Command / concept	Purpose and notes
`getent passwd alex; getent group IT`	Check whether the account and group already exist. The semicolon runs the second command after the first.
`sudo groupadd IT`	Create the IT group. sudo runs the administrative command with elevated privileges.
`getent group IT`	Look up the group in the system account databases. The observed entry was IT:x:1003: before users were added.
`sudo useradd -m -s /bin/bash alex`	Create Alex's account. -m creates a home directory; -s sets the login shell.
`getent passwd alex`	Display Alex's account record. Observed format: name:password-placeholder:UID:GID:GECOS:home:shell.
`sudo usermod -aG IT alex`	Add Alex to supplementary group IT. -a appends; -G specifies supplementary groups. Using -G without -a can replace existing supplementary group memberships.
`id alex`	Show Alex's UID, primary GID, and group memberships. Used to verify alex as primary group and IT as supplementary group.

Command / concept	Purpose and notes
`getent group alex IT`	Look up both group records. The primary group may not list its user as a supplementary member in /etc/group.

Alex's observed account record was `alex:x:1003:1004::/home/alex:/bin/bash`. This showed UID 1003, primary GID 1004, home directory /home/alex, and Bash as the shell.

The home directory was observed with mode 750 (`drwxr-x---`). This permits the owner to read, write, and enter it; the owning group can read and enter; others have no access.

4. Passwords and switching users

Command / concept	Purpose and notes
`sudo passwd alex`	Set or change Alex's password. The password is entered interactively and should never be shared or placed in the portfolio.
`su alex`	Switch to Alex without starting a full login environment. The current working directory may remain unchanged.
`su - alex`	Start a login shell as Alex. The dash loads the login environment and changes to Alex's home directory.
`su - maria`	Start a login shell as Maria. Used to test file creation as Maria.
`su - punith2205`	Return to the original account using a login shell. Requires the target account's password when invoked this way by another user.
`exit`	Leave the current shell or user session. Nested su sessions may require more than one exit.
`pwd`	Print the current working directory. It reports location; it does not change directories. Use cd to change location.

5. Creating Maria and adding her to IT

Command / concept	Purpose and notes
`sudo adduser maria`	Create Maria interactively. The command prompts for a password and optional account information.
`sudo adduser maria IT`	Add the existing user Maria to the existing IT group. This form was run after Maria's account had been created.
`id`	Show the current user's identity and group memberships. When logged in as Maria, it confirmed her primary group and supplementary IT membership.
`getent group IT`	Verify the IT group membership list. Final observed result: IT:x:1003:alex,maria.

The attempt to add Maria before creating her account failed because the user did not yet exist. The correct sequence in this project was to create the account first, then add it to IT.

6. Shared directory: ownership and permissions

Command / concept	Purpose and notes
`sudo mkdir -p /srv/it-share`	Create the shared directory. The /srv location is commonly used for data served or provided by the system.
`ls -ld /srv/it-share`	Display the directory itself, not its contents. -d is useful for inspecting a directory's own permissions and ownership.

Command / concept	Purpose and notes
`sudo chown root:IT /srv/it-share`	Set owner to root and group to IT. chown can change owner and group.
`sudo chmod 770 /srv/it-share`	Give owner and group rwx; deny all permissions to others. 7 = rwx; 0 = no permissions.
`sudo chown alex:IT /srv/it-share`	Change the directory owner to Alex and keep group IT. Final owner:group became alex:IT.
`sudo chmod g+s /srv/it-share`	Enable setgid on the directory. New files and subdirectories inherit the directory's group.
`ls -ld /srv/it-share`	Verify the final directory mode and ownership. Observed: drwxrws--- ... alex IT ... /srv/it-share.

The final symbolic mode was `drwxrws---`. Its numeric mode is `2770`: the leading 2 enables setgid, followed by 770 for owner/group/others.

The `s` appears in the group execute position because setgid is enabled and the group execute bit is also set.

7. Files, editing, and collaboration

Command / concept	Purpose and notes
`touch ~/company-notes.txt`	Create an empty file in the current user's home directory, or update its timestamp if it exists. The tilde (~) expands to the current user's home directory.
`touch /srv/it-share/alex-notes.txt`	Create a file in the shared directory. An absolute path avoids confusion about the current directory.
`touch /srv/it-share/it-test.txt`	Create a shared test file as Alex. Created after setgid was enabled, so its group was IT.
`touch /srv/it-share/maria-notes.txt`	Create Maria's shared notes file. Created by Maria after setgid was enabled; group inherited as IT.
`nano company-notes.txt`	Open a text file in the Nano editor. The path is relative to the current working directory.
`nano /srv/it-share/alex-notes.txt`	Edit Alex's notes in the shared directory. An absolute path targets the shared file.
`cat company-notes.txt`	Print a file's contents to the terminal. Useful for checking a small text file.
`cat /srv/it-share/maria-notes.txt`	Read Maria's shared notes. Used to verify that Alex's appended line appeared.
`echo 'text' >> /path/to/file`	Append text to the end of a file. >> appends; it does not replace existing content.
`ls -l /srv/it-share`	List directory contents with ownership and permissions. Showed alex-notes.txt, it-test.txt, and maria-notes.txt.

The collaboration test showed that Alex could append a line to Maria's file. This worked because the file belonged to group IT and had group-write permission, while Alex was a member of IT.

A file created before setgid was enabled retained its earlier group. In this project, alex-notes.txt remained owned by group alex, while files created later inherited group IT.

8. Inspecting ownership, permissions, and access

Command / concept	Purpose and notes
`ls -ld /home/alex`	Inspect the home directory's own metadata. Does not list the directory's contents.

Command / concept	Purpose and notes
`ls -l /home/alex`	List the contents of Alex's home directory. This failed for punith2205 because the directory permissions did not allow access.
`sudo ls -l /home/alex`	List the same directory with elevated privileges. Worked because sudo supplied administrative access.
`stat -c '%A %a %U %G %n' /home/alex`	Print symbolic mode, numeric mode, owner, group, and path. Format fields: %A mode; %a octal mode; %U owner; %G group; %n name.
`stat -c '%A %a %U %G %n' /home/alex/company-notes.txt`	Inspect file mode and ownership. Access can fail if the caller cannot traverse a parent directory.
`sudo stat -c '%A %a %U %G %n' /home/alex/company-notes.txt`	Inspect the file with elevated privileges. Used when ordinary access was denied.
`id punith2205`	Display the original user's group memberships. Showed that sudo membership does not automatically make every ordinary file operation privileged.
`sudo -i`	Open an interactive root login shell. Use carefully; exit to return to the previous account.

A directory's execute bit means permission to traverse/search it. Read permission allows listing its names. Therefore, being able to inspect a directory's own metadata with `ls -ld` or `stat` does not necessarily mean you can list its contents.

9. Finding command help and syntax

Command / concept	Purpose and notes
`man -k adduser`	Search manual-page names and descriptions for a keyword. Equivalent to apropos in many Linux distributions.
`man adduser`	Open the manual page for adduser. Press q to quit; use /word to search and n to find the next match.
`man 8 adduser`	Open section 8 of the manual for adduser. Section 8 contains system-administration commands.
`adduser --help`	Request a command's built-in help summary. Useful for a quick syntax overview when supported.
`type adduser`	Show how Bash resolves the command name. Can identify aliases, functions, builtins, or executable paths.

During the project, `man -k adduser` showed relevant manual entries, including adduser(8) and adduser.conf(5). The help commands above are useful reference suggestions; they were not all executed during the project.

10. Mistakes and what they taught me

Command / concept	Purpose and notes
`chuser ...`	Command not found. The intended command was chown. chown changes ownership; chmod changes permissions; chgrp changes group ownership.
`ls - ld ...`	ls treated '-' and 'ld' as separate path arguments. Options must be written together as -ld.
`pwd /home/alex`	pwd printed the current directory rather than changing to the supplied path. Use cd /home/alex to change directories.
`nano alex-notes.txt`	Opened or created a file relative to the current directory. Use an absolute path when you intend to edit a file in /srv/it-share.

Command / concept	Purpose and notes
`echo 'message' /path/to/file`	Printed both arguments; it did not write to the file. Use > or >> for redirection.
`sudo adduser maria IT`	Failed when Maria did not yet exist. Create the account first, then add it to the group.
`adduser man / adduser help`	The words were interpreted as usernames rather than help requests. Use the manual page or a supported --help option.

11. Redirection and permission quick reference

Command / concept	Purpose and notes
`>`	Redirect output and overwrite the target file. Be careful: existing contents are replaced.
`>>`	Redirect output and append to the target file. Used in the collaboration test.
`r = 4`	Read permission. For a directory, read permits listing names.
`w = 2`	Write permission. For a directory, write permits creating/removing/renaming entries when combined with execute.
`x = 1`	Execute permission. For a directory, execute permits traversal/search.
`chmod 770 DIR`	Owner rwx, group rwx, others none. Numeric mode for a shared directory before adding setgid.
`chmod 2770 DIR`	Same permissions plus setgid. Equivalent to enabling g+s on a 770 directory.
`chown USER:GROUP PATH`	Change owner and group. Requires appropriate administrative privileges for system paths.

12. Final verification checklist

• The IT group exists.

• Both alex and maria are members of IT.

• The shared directory is owned by alex:IT.

• The shared directory mode is 2770 (drwxrws---).

• The setgid bit is enabled.

• Files created after setgid was enabled inherited group IT.

• Alex and Maria successfully collaborated by appending to a shared file.

• The project README was created and its contents were checked.

Final observed group entry: `IT:x:1003:alex,maria`

Final observed shared-directory listing: alex-notes.txt (alex:alex), it-test.txt (alex:IT), maria-notes.txt (maria:IT). The files had mode 664 in the observed listing.

Learning note: File modes and group inheritance work together. Setgid determines the inherited group for new items in the directory; the file-creation mode and umask determine the permissions on each new file.